

EXERCICES PYTHON

`class - def __init__ - self`

10 exercices progressifs avec corrections

* Exercices 1 a 4 - Facile
** Exercices 5 a 8 - Intermediaire
*** Exercices 9 et 10 - Difficile

Exercice 1 - Creer ta premiere class

* Facile

Objectif : Apprendre a ecrire la structure minimale d une class Python.

A faire Ce que tu dois faire :

Cree une class qui s appelle Chat
Ajoute un `__init__` qui accepte un parametre : nom
Dans `__init__`, stocke le nom dans l objet avec `self.nom`
Cree un objet `chat1` avec le nom Whiskers
Affiche le nom du chat avec `print()`

Code de depart

```
# Complete ce code :  
class _____:  
    def _____(self, nom):  
        self.____ = ____  
  
chat1 = Chat(_____)   
print(_____)
```

Indice Indice :

`self.nom = nom` stocke le parametre nom dans l objet. Pour afficher : `print(chat1.nom)`

Pieges Pieges a eviter :

N oublie pas le : apres `class Chat` et apres `def __init__(self, nom)`
`__init__` s ecrit avec DEUX underscores avant et apres, pas un seul

Ta reponse

Correction

```
class Chat:
    def __init__(self, nom): # constructeur avec parametre nom
        self.nom = nom      # on stocke le nom dans l objet

chat1 = Chat('Whiskers')    # cree un objet Chat, __init__ s execute
print(chat1.nom)           # affiche : Whiskers
```

Exercice 2 - Plusieurs attributs

* Facile

Objectif : Pratiquer le stockage de plusieurs informations dans un objet.

A faire Ce que tu dois faire :

Cree une class Personne

Le `__init__` doit accepter : prenom, age, ville

Stocke les 3 informations avec self.

Cree deux personnes : Alice (25 ans, Paris) et Bob (30 ans, Lyon)

Affiche pour chaque personne : Alice a 25 ans et habite a Paris

Code de depart

```
class Personne:
    def __init__(self, ____, ____, ____):
        self.____ = ____
        self.____ = ____
        self.____ = ____

alice = Personne(____, ____, ____ )
bob   = Personne(____, ____, ____ )

print(f'{alice.____} a {alice.____} ans et habite a {alice.____} ')
print(f'{bob.____} ...')
```

Indice Indice :

Utilise les f-strings : `f'{self.prenom} a {self.age} ans'` - les accolades `{}` permettent d insérer des variables

Pieges Pieges a eviter :

Chaque attribut doit avoir son propre self. : `self.prenom`, `self.age`, `self.ville`

L ordre des parametres compte : `Personne('Alice', 25, 'Paris')` doit correspondre a `def __init__(self, prenom, age, ville)`

Ta reponse

Correction

```
class Personne:
    def __init__(self, prenom, age, ville):
        self.prenom = prenom
        self.age     = age
        self.ville   = ville

alice = Personne('Alice', 25, 'Paris')
bob   = Personne('Bob',   30, 'Lyon')

print(f'{alice.prenom} a {alice.age} ans et habite a {alice.ville}')
# affiche : Alice a 25 ans et habite a Paris
print(f'{bob.prenom} a {bob.age} ans et habite a {bob.ville}')
```

Exercice 3 - Ajouter une methode

* Facile

Objectif : Apprendre a ajouter une methode (une action) a une class.

A faire Ce que tu dois faire :

Cree une class Chien

Le `__init__` accepte : nom, race

Ajoute une methode `aboyer(self)` qui affiche : `Nom_du_chien dit : Woof !`

Cree un chien nomme Rex de race Labrador

Appelle la methode `aboyer()` sur ton objet

Code de depart

```
class Chien:
    def __init__(self, nom, race):
        self.nom = nom
        self.race = race

    def aboyer(self):
        # ne pas oublier le parametre !
        print(f'{self.nom} dit : Woof !')

rex = Chien('Rex', 'Labrador')
rex.aboyer() # appelle la methode
```

Indice Indice :

Une methode, c est comme une fonction, mais DANS une class.

Elle prend toujours `self` en 1er parametre.

Pour appeler une methode sur un objet : `objet.methode()`

Pieges Pieges a eviter :

Oublier `self` dans `def aboyer(self)`: est l erreur la plus frequente !

Quand tu APPELLES la methode (`rex.aboyer()`), tu ne mets PAS `self` - Python le fait seul

Ta reponse

Correction

```
class Chien:
    def __init__(self, nom, race):
        self.nom = nom
        self.race = race

    def aboyer(self):
        # self toujours en 1er
        print(f'{self.nom} dit : Woof !') # on accede au nom via self

rex = Chien('Rex', 'Labrador')
rex.aboyer() # affiche : Rex dit : Woof !
```

Exercice 4 - Methode avec parametre

* Facile

Objectif : Apprendre qu'une methode peut avoir des parametres en plus de self.

A faire Ce que tu dois faire :

Cree une class Calculatrice

Le `__init__` ne prend que self (pas d'autres parametres)

Ajoute une methode `additionner(self, a, b)` qui affiche le resultat de $a + b$

Ajoute une methode `multiplier(self, a, b)` qui affiche le resultat de $a * b$

Cree une calculatrice et teste les deux methodes

Code de depart

```
class Calculatrice:
    def __init__(self):          # pas de parametre special ici
        pass                   # pass = rien a faire dans __init__

    def additionner(self, __, __):
        print(f'{__} + {__} = {__}')

    def multiplier(self, __, __):
        print(__)

calc = Calculatrice()
calc.additionner(__, __)
calc.multiplier(__, __)
```

Indice Indice :

`f'{a} + {b} = {a + b}'` - tu peux faire des calculs directement dans les accolades d'un f-string

Pieges Pieges a eviter :

self est TOUJOURS le 1er parametre, meme quand il y en a d'autres : `def additionner(self, a, b)`

Quand tu appelles `calc.additionner(3, 5)`, tu ne passes PAS self - juste les vraies valeurs

Ta reponse

Correction

```
class Calculatrice:
    def __init__(self):
        pass

    def additionner(self, a, b):
        print(f'{a} + {b} = {a + b}')

    def multiplier(self, a, b):
        print(f'{a} x {b} = {a * b}')

calc = Calculatrice()
calc.additionner(3, 5)      # affiche : 3 + 5 = 8
calc.multiplier(4, 6)      # affiche : 4 x 6 = 24
```

Exercice 5 - Modifier un attribut

**** Intermediaire**

Objectif : Comprendre comment modifier un attribut apres la creation de l objet.

A faire Ce que tu dois faire :

Cree une class CompteBancaire

Le `__init__` accepte : titulaire et solde_initial

Ajoute une methode `deposer(self, montant)` qui augmente le solde

Ajoute une methode `retirer(self, montant)` qui diminue le solde

Ajoute une methode `afficher_solde(self)` qui affiche le solde actuel

Teste avec : depot de 500, retrait de 200, affichage

Code de depart

```
class CompteBancaire:
    def __init__(self, titulaire, solde_initial):
        self.titulaire = titulaire
        self.solde = solde_initial

    def deposer(self, montant):
        self.solde = self.solde + ____
        print(f'Depot de {montant}. Nouveau solde : {self.____}')

    def retirer(self, montant):
        self.solde = ____
        print(____)

    def afficher_solde(self):
        print(____)

compte = CompteBancaire('Alice', 1000)
compte.deposer(____)
compte.retirer(____)
compte.afficher_solde()
```

Indice Indice :

`self.solde += montant` est un raccourci pour `self.solde = self.solde + montant`

L attribut `self.solde` est partage entre toutes les methodes grace a `self`

Pieges Pieges a eviter :

Si tu ecris `solde += montant` sans `self.`, Python cree une variable locale qui disparaît ensuite

Toujours utiliser `self.solde` pour modifier le vrai solde de l objet

Ta reponse

Correction

```
class CompteBancaire:
    def __init__(self, titulaire, solde_initial):
        self.titulaire = titulaire
        self.solde = solde_initial

    def deposer(self, montant):
        self.solde += montant
        print(f'Depot de {montant}. Nouveau solde : {self.solde}')

    def retirer(self, montant):
        self.solde -= montant
        print(f'Retrait de {montant}. Nouveau solde : {self.solde}')

    def afficher_solde(self):
        print(f'Solde de {self.titulaire} : {self.solde}')

compte = CompteBancaire('Alice', 1000)
compte.deposer(500)      # Depot de 500. Nouveau solde : 1500
compte.retirer(200)      # Retrait de 200. Nouveau solde : 1300
compte.afficher_solde()  # Solde de Alice : 1300
```

Exercice 6 - Trouver les erreurs

**** Intermediaire**

Objectif : Savoir identifier et corriger les erreurs classiques de POO.

A faire Ce que tu dois faire :

Le code ci-dessous contient 5 erreurs.

Identifie chacune d'elles et reecris le code corrige.

Les erreurs concernent class, __init__, self et l'indentation.

Code de depart

```
# CODE AVEC ERREURS - trouve et corrige les 5 erreurs !

class voiture:                                # erreur 1
    def __init__(self, marque):               # erreur 2
        self.marque = marque                  # erreur 3

    def demarrer():                            # erreur 4
        print(marque + ' demarre !')         # erreur 5

ma_voiture = Voiture('Renault')
ma_voiture.demarrer()
```

Indice Indice :

Erreur 1 : le nom d'une class commence par une Majuscule

Erreur 2 : compter les underscores de __init__

Erreur 3 : regarder l'indentation (nombre d'espaces)

Erreur 4 : il manque un parametre obligatoire dans demarrer()

Erreur 5 : comment accede-t-on a un attribut depuis une methode ?

Pieges Pieges a eviter :

Il y a exactement 5 erreurs, pas plus, pas moins.

Chaque erreur est independante des autres.

Ta reponse

Correction

```
class Voiture:                                # Majuscule a Voiture
    def __init__(self, marque):                # deux underscores de chaque cote
        self.marque = marque                  # indente de 8 espaces

    def demarrer(self):                        # self en parametre
        print(self.marque + ' demarre !')     # self.marque et non juste marque

ma_voiture = Voiture('Renault')
ma_voiture.demarrer()                         # affiche : Renault demarre !
```

Exercice 7 - Deux objets qui interagissent

**** Intermediaire**

Objectif : Créer plusieurs objets d'une même classe et les utiliser ensemble.

A faire Ce que tu dois faire :

Crée une classe Joueur

Le `__init__` accepte : nom et points (valeur par défaut : 0)

Ajoute une méthode `gagner_points(self, nombre)` qui ajoute des points

Ajoute une méthode `afficher_score(self)` qui affiche le score

Crée deux joueurs, fais-les gagner des points différents

Affiche lequel a le meilleur score avec une comparaison

Code de depart

```
class Joueur:
    def __init__(self, nom, points=0): # points=0 = valeur par défaut
        self.nom = nom
        self.points = points

    def gagner_points(self, nombre):
        self.points += nombre

    def afficher_score(self):
        print(f"Le joueur {self.nom} a {self.points} points")

joueur1 = Joueur('Alice')
joueur2 = Joueur('Bob')
joueur1.gagner_points(150)
joueur2.gagner_points(200)
joueur1.gagner_points(80)
joueur1.afficher_score()
joueur2.afficher_score()
if joueur1.points > joueur2.points:
    print(f'{joueur1.nom} gagne !')
else:
    print(f'{joueur2.nom} gagne !')
```

Indice Indice :

`points=0` signifie que si tu ne passes pas de valeur, `points` vaudra 0 par défaut.

Pieges Pieges a éviter :

`joueur1.points` et `joueur2.points` sont INDEPENDANTS - chaque objet a son propre compteur

Pour comparer les scores : `joueur1.points > joueur2.points`

Ta reponse

Correction

```
class Joueur:
    def __init__(self, nom, points=0):
        self.nom = nom
        self.points = points

    def gagner_points(self, nombre):
        self.points += nombre

    def afficher_score(self):
        print(f'{self.nom} : {self.points} points')

joueur1 = Joueur('Alice')
joueur2 = Joueur('Bob')
joueur1.gagner_points(150)
joueur2.gagner_points(200)
joueur1.gagner_points(80)
joueur1.afficher_score()    # Alice : 230 points
joueur2.afficher_score()    # Bob : 200 points
if joueur1.points > joueur2.points:
    print(f'{joueur1.nom} gagne !')    # Alice gagne !
```

Exercice 8 - Fenetre Tkinter avec class

**** Intermediaire**

Objectif : Reproduire la structure de ton cours - une vraie app Tkinter avec class.

A faire Ce que tu dois faire :

Cree une class Application
Dans `__init__`, cree une fenetre 500x400 avec le titre Mon Application
Ajoute un bouton `tk.Button` avec le texte Cliquez-moi !
Ajoute une methode `action(self)` appelee quand on clique le bouton
La methode `action()` doit afficher un message dans le terminal
Lance l application avec `root.mainloop()`

Code de depart

```
import tkinter as tk

class Application:
    def __init__(self, root):
        self.root = root
        self.root.title('Mon Application')
        self.root.geometry('500x400')

        self.bouton = tk.Button(
            self.root,
            text='Cliquez-moi !',
            command=self.action  # methode a appeler au clic
        )
        self.bouton.pack()

    def action(self):
        print('Cliquez-moi !')

root = tk.Tk()
app = Application(root)
root.mainloop()
```

Indice Indice :

`command=self.action` sans parentheses - on passe la reference a la methode
`self.bouton = tk.Button(...)` permet d acceder au bouton plus tard si besoin

Pieges Pieges a eviter :

`command=self.action()` AVEC des parentheses appelle la fonction immediatement - mauvais !
`command=self.action` SANS parentheses passe la reference - correct !

Ta reponse

Correction

```
import tkinter as tk

class Application:
    def __init__(self, root):
        self.root = root
        self.root.title('Mon Application')
        self.root.geometry('500x400')

        self.bouton = tk.Button(
            self.root,
            text='Cliquez-moi !',
            command=self.action    # reference SANS () - important !
        )
        self.bouton.pack()

    def action(self):
        print('Bouton clique !')

root = tk.Tk()
app = Application(root)
root.mainloop()
```

Exercice 9 - Class avec liste d attributs

*** Difficile

Objectif : Gerer une liste comme attribut d un objet - concept tres frequent en programmation.

A faire Ce que tu dois faire :

Cree une class ListeCourses

Le `__init__` initialise une liste vide : `self.articles = []`

Ajoute une methode `ajouter(self, article)` qui ajoute un article

Ajoute une methode `supprimer(self, article)` qui retire un article

Ajoute une methode `afficher(self)` qui affiche tous les articles numerotes

Teste avec 4 articles, puis supprimes-en un

Code de depart

```
class ListeCourses:
    def __init__(self):
        self.articles = [] # liste vide au depart

    def ajouter(self, article):
        self.____.append(____) # .append() ajoute a la fin d une liste
        print(f'{article} ajoute !')

    def supprimer(self, article):
        if article in self.____:
            self.____.remove(____)
        else:
            print(f'{article} non trouve')

    def afficher(self):
        for i, article in enumerate(self.____, start=1):
            print(f'{i}. {article}')

liste = ListeCourses()
liste.ajouter('Pain')
liste.ajouter('Lait')
liste.ajouter('Oeufs')
liste.ajouter('Beurre')
liste.afficher()
liste.supprimer('Lait')
liste.afficher()
```

Indice Indice :

`self.articles = []` initialise une liste Python vide

`liste.append(element)` ajoute un element a la fin

`liste.remove(element)` supprime la premiere occurrence

`enumerate(liste, start=1)` donne l index et la valeur en meme temps

Pieges Pieges a eviter :

Ne pas oublier self. devant articles dans toutes les methodes
self.articles et articles sont deux choses differentes

Ta reponse

Correction

```
class ListeCourses:
    def __init__(self):
        self.articles = []

    def ajouter(self, article):
        self.articles.append(article)
        print(f'{article} ajoute !')

    def supprimer(self, article):
        if article in self.articles:
            self.articles.remove(article)
            print(f'{article} supprime !')
        else:
            print(f'{article} non trouve')

    def afficher(self):
        print('--- Ma liste de courses ---')
        for i, article in enumerate(self.articles, start=1):
            print(f'{i}. {article}')

liste = ListeCourses()
liste.ajouter('Pain')
liste.ajouter('Lait')
liste.ajouter('Oeufs')
liste.ajouter('Beurre')
liste.afficher()          # 1.Pain 2.Lait 3.Oeufs 4.Beurre
liste.supprimer('Lait')
liste.afficher()          # 1.Pain 2.Oeufs 3.Beurre
```

Exercice 10 - Projet - Application Tkinter Compteur

*** Difficile

Objectif : Créer une mini application Tkinter avec plusieurs éléments et méthodes - exercice de synthèse.

A faire Ce que tu dois faire :

Crée une class ApplicationCompteur
Le `__init__` crée une fenêtre 300x200 Mon Compteur
Dans `__init__`, crée un attribut `self.compteur = 0`
Affiche le compteur dans la fenêtre avec un `tk.Label`
Ajoute un bouton +1 qui appelle une méthode `incrémenter(self)`
Ajoute un bouton Reinitialiser qui appelle `reinitialiser(self)`
`incrémenter()` ajoute 1 et met à jour le Label
`reinitialiser()` remet à 0 et met à jour le Label

Code de départ

```
import tkinter as tk

class ApplicationCompteur:
    def __init__(self, root):
        self.root = root
        self.compteur = 0
        self.root.title(____)
        self.root.geometry(____)

        self.label = tk.Label(root, text='0', font=('Arial', 48))
        self.label.pack(pady=20)

        self.btn_plus = tk.Button(root, text='+1', command=self.___)
        self.btn_plus.pack(side='left', padx=20)

        self.btn_reset = tk.Button(root, text='Reinitialiser', command=self.___)
        self.btn_reset.pack(side='right', padx=20)

    def incrémenter(self):
        self.compteur += ____
        self.label.config(text=____(self.compteur))

    def reinitialiser(self):
        self.compteur = ____
        self.label.config(text=____)

root = tk.Tk()
app = ApplicationCompteur(root)
root.mainloop()
```

Indice Indice :

tk.Label(root, text='0') cree un texte dans la fenetre
label.config(text='nouveau texte') met a jour le texte d un Label existant
Stoque le label dans self.label pour y acceder dans les methodes

Pieges Pieges a eviter :

Ne pas oublier de stocker le Label dans self.label (et non juste label)
Pour mettre a jour l'affichage : self.label.config(text=str(self.compteur))
str() est necessaire car le compteur est un entier et config attend du texte

Ta reponse

Correction

```
import tkinter as tk

class ApplicationCompteur:
    def __init__(self, root):
        self.root = root
        self.compteur = 0
        self.root.title('Mon Compteur')
        self.root.geometry('300x200')

        self.label = tk.Label(root, text='0', font=('Arial', 48))
        self.label.pack(pady=20)

        self.btn_plus = tk.Button(root, text='+1', command=self.incrementer)
        self.btn_plus.pack(side='left', padx=20)

        self.btn_reset = tk.Button(root, text='Reinitialiser',
command=self.reinitialiser)
        self.btn_reset.pack(side='right', padx=20)

    def incrementer(self):
        self.compteur += 1
        self.label.config(text=str(self.compteur))

    def reinitialiser(self):
```

```
        self.compteur = 0
        self.label.config(text='0')

root = tk.Tk()
app = ApplicationCompteur(root)
root.mainloop()
```